

Projet de week-end — Gestion des réservations de salles universitaires

Niveau : intermédiaire

Durée : un week-end

Technologie : PHP orienté objet

Base de données : MySQL

Travail : individuel

A. Description du Projet

1. Contexte

L'université possède plusieurs salles utilisées pour :

- les cours ;
- les soutenances ;
- les réunions ;
- les travaux pratiques ;
- les événements étudiants.

La réservation est actuellement réalisée par courriel. Cela provoque des doublons, car plusieurs personnes peuvent demander la même salle au même horaire.

Vous devez développer une application web permettant de consulter les salles et de gérer leurs réservations.

L'application sera développée sans framework complet, mais avec des composants spécialisés installés avec Composer.

2. Objectifs pédagogiques

À la fin du projet, l'étudiant devra savoir :

- structurer une application PHP en couches ;
- recevoir et traiter une requête HTTP ;
- utiliser un routeur externe ;
- valider les données d'un formulaire ;
- manipuler une base avec un ORM ;
- créer des relations entre modèles ;

- placer les règles métier dans des services ;
 - injecter les dépendances ;
 - configurer un conteneur d'injection ;
 - appliquer plusieurs principes SOLID ;
 - versionner progressivement une application avec Git.
-

3. Dépendances imposées

Les bibliothèques suivantes doivent être utilisées :

Besoin	Dépendance
Routeur	<code>nikic/fast-route</code>
Validation	<code>respect/validation</code>
ORM	<code>illuminate/database</code>
Conteneur	<code>php-di/php-di</code>
Variables d'environnement	<code>vlucas/phpdotenv</code>

- **FastRoute** associe les méthodes et chemins HTTP à des handlers, avec gestion des paramètres dynamiques ainsi que des réponses 404 et 405. [Documentation FastRoute](#)
- **Eloquent** est fourni par `illuminate/database` et peut être employé en dehors de Laravel grâce à `Capsule\Manager`. Il propose un ORM Active Record, un Query Builder et un Schema Builder. [Documentation Illuminate Database](#)
- **PHP-DI** peut construire automatiquement les classes à partir de leurs constructeurs et de définitions explicites. Sa documentation recommande de limiter l'accès direct au conteneur au point d'entrée de l'application. [Documentation PHP-DI](#)
- **Respect\Validation** fournit des validateurs composables pour contrôler les données d'un formulaire. [Documentation Respect\Validation](#)

Installation indicative

Pour un environnement PHP 8.2 ou 8.3 :

```
composer require \  
    nikic/fast-route \  
    respect/validation:^2.4 \  
    illuminate/database:^12.0 \  
    php-di/php-di:^7.0 \  
    vlucas/phpdotenv
```

4. Fonctionnalités attendues

Gestion des salles

L'utilisateur doit pouvoir :

- consulter la liste des salles ;
- afficher le détail d'une salle ;
- ajouter une salle ;
- modifier une salle ;
- activer ou désactiver une salle.

Salle(id,nom,batiment,capacite,type,active,created_at,updated_at)

Types autorisés :cours|informatique|laboratoire|amphitheatre|reunion)

Gestion des réservations

L'utilisateur doit pouvoir :

- consulter toutes les réservations ;
- filtrer les réservations par salle ;
- afficher une réservation ;
- créer une réservation ;
- annuler une réservation.

réservation

(id,salle_id,responsable,email,motif,date_debut,date_fin,statut,created_at,updated_at)

Statuts autorisés :confirmée,annulée

5. Règles métier

Une réservation est acceptée uniquement si :

1. la salle existe ;
2. la salle est active ;
3. le nom du responsable est renseigné ;
4. l'adresse électronique est valide ;

5. le motif contient entre 5 et 255 caractères ;
6. la date de début précède la date de fin ;
7. la réservation dure au maximum quatre heures ;
8. la réservation commence dans le futur ;
9. aucune réservation confirmée ne chevauche cette période.

Exemple :

Deux réservations se chevauchent lorsque :

***nouveauDebut < reservationExistante.dateFin
ET
nouvelleFin > reservationExistante.dateDebut***

Exemple :

Réservation existante : 10h00 → 12h00

Nouvelle réservation : 11h00 → 13h00

Résultat : conflit

Une réservation annulée ne bloque plus la salle.

6. Contraintes architecturales

Les étudiants doivent respecter les contraintes suivantes :

- `public/index.php` est l'unique point d'entrée ;
 - les routes sont déclarées dans un fichier séparé ;
 - les contrôleurs ne réalisent aucune requête ORM ;
 - les modèles Eloquent ne lisent pas `$_POST` ;
 - les vues ne contiennent aucune règle métier ;
 - les règles de disponibilité sont placées dans un service ;
 - les données HTTP sont validées avant leur utilisation ;
 - les dépendances sont reçues dans les constructeurs ;
 - seul le point d'entrée récupère directement un objet dans le conteneur ;
 - les mots de passe et paramètres sensibles ne sont pas versionnés.
-

7. Arborescence attendue

```
reservation-salles/
├── config/
│   ├── container.php
│   └── database.php
├── database/
│   ├── migrations/
│   └── seed.php
├── public/
│   ├── assets/
│   │   └── style.css
│   └── index.php
├── routes/
│   └── web.php
├── src/
│   ├── Controller/
│   ├── DTO/
│   ├── Exception/
│   ├── Model/
│   ├── Repository/
│   ├── Service/
│   ├── Validation/
│   └── View/
├── templates/
│   ├── error/
│   ├── layout/
│   ├── reservation/
│   └── salle/
├── tests/
│   ├── Unit/
│   └── Integration/
├── .env.example
├── .gitignore
├── CHANGELOG.md
├── composer.json
├── composer.lock
├── phpunit.xml
└── README.md
```

Il appartient aux étudiants de justifier cette organisation.

8. Modèle de données

Il appartient aux étudiants de définir le Diagramme de classe et d'en déduire le schéma de base de données .

B. Déroulement du projet

Étape 0 — Initialiser le dépôt

Travail demandé

1. Initialisez Git.
 2. Créez la branche `main`.
 3. Préparez `.gitignore`.
 4. Créez `README.md` et `CHANGELOG.md`.
 5. Créez le premier commit.
 6. Version attendue : v0.0.0
-

Étape 1 — Initialiser le projet Composer

Travail demandé

1. Créez `composer.json`.
2. Configurez l'autoloading PSR-4.
3. Installez les dépendances.
4. Créez l'arborescence.
5. Vérifiez que Composer charge une classe `App\Application`.

Questions

1. Quel est le rôle de Composer ?
2. Quelle différence existe entre `require` et `require-dev` ?
3. Pourquoi faut-il versionner `composer.lock` ?
4. Pourquoi ne versionne-t-on pas `vendor/` ?

Versionnement

Branche : feature/01-composer

Tag : v0.1.0

Commits possibles :

init: créer le projet PHP

chore: configurer l'autoloading

chore: installer les dépendances

Étape 2 — Configurer Eloquent

Travail demandé

1. Ajoutez `.env.example`.
2. Chargez les variables d'environnement.
3. Configurez `Capsule\Manager`.
4. Démarrez Eloquent.
5. Vérifiez la connexion.
6. Créez les tables.

Variables attendues :

APP_ENV=development

APP_DEBUG=true

DB_DRIVER=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=reservation_salles

DB_USERNAME=root

DB_PASSWORD=

Contraintes

- `.env` ne doit pas être versionné ;
- aucune classe métier ne doit appeler `getenv()` ;
- la connexion doit être configurée une seule fois ;
- les erreurs de connexion doivent être gérées.

Questions

1. Quel rôle joue `Capsule\Manager` ?
2. Pourquoi Eloquent peut-il fonctionner sans Laravel ?
3. Où doit se trouver le démarrage de l'ORM ?
4. Quelle différence existe entre ORM et SQL écrit à la main ?

Versionnement

Branche : feature/02-eloquent

Étape 3 — Créer les modèles

Travail demandé

Créez :

App\Model\Salle

App\Model\Reservation

Configurez :

- le nom des tables ;
- les propriétés assignables ;
- les conversions de types ;
- les dates ;
- la relation entre les deux modèles.

Relations attendues :

Une salle possède plusieurs réservations

Une réservation appartient à une salle

Le modèle **Salle** doit permettre :

\$salle->reservations;

Le modèle **Reservation** doit permettre :

\$reservation->salle;

Questions

1. Quel type de relation Eloquent avez-vous utilisé ?
2. Pourquoi déclarer \$fillable ou \$guarded ?
3. Pourquoi convertir **active** en booléen ?
4. Pourquoi convertir les dates en objets ?

Versionnement

Branche : feature/03-modeles

Tag : v0.3.0

Étape 4 — Ajouter les données initiales

Travail demandé

Créez un script permettant d'ajouter au moins cinq salles :

Amphithéâtre A — 250 places

Salle B12 — 40 places

Laboratoire Chimie — 24 places

Salle Informatique 1 — 30 places

Salle de réunion — 12 places

Le script doit pouvoir être exécuté plusieurs fois sans créer inutilement de doublons.

Questions

1. Quelle différence existe entre migration et seeder ?
2. Pourquoi les données initiales doivent-elles être reproductibles ?
3. Comment empêcher les doublons ?

Versionnement

Branche : feature/04-donnees-initiales

Tag : v0.4.0

Étape 5 — Créer la validation

Travail demandé

Créez un contrat commun :

interface ValidatorInterface

```
{  
    public function validate(array $data): ValidationResult;  
}
```

Créez ensuite :

- ***SalleValidator***
- ***ReservationValidator***
- ***ValidationResult***

Les validateurs doivent utiliser Respect\Validation.

ValidationResult devra permettre de connaître :

- si les données sont valides ;
- les erreurs associées à chaque champ ;
- les données acceptées.

Exemple d'utilisation souhaité :

```
$resultat = $validator->validate($data);
```

```
if (!$resultat->isValid()) {  
    $errors = $resultat->errors();  
}
```

Validation d'une salle

- **nom** : obligatoire, 2 à 100 caractères ;
- **batiment** : obligatoire, 2 à 100 caractères ;
- **capacite** : entier compris entre 1 et 1 000 ;
- **type** : valeur autorisée ;
- **active** : booléen.

Validation d'une réservation

- **salle_id** : entier positif ;
- **responsable** : 2 à 120 caractères ;
- **email** : adresse valide ;
- **motif** : 5 à 255 caractères ;
- **date_debut** : date valide ;
- **date_fin** : date valide.

La comparaison entre les dates sera effectuée dans la couche métier.

Questions

1. Pourquoi séparer la validation syntaxique des règles métier ?
2. Pourquoi créer une interface de validation ?
3. Pourquoi le validateur ne doit-il pas enregistrer les données ?
4. Comment retourner plusieurs erreurs en une seule fois ?

Versionnement

Branche : feature/05-validation

Tag : v0.5.0

Étape 6 — Créer les objets de transport

Travail demandé

Créez :

- CreerSalleDTO
- CreerReservationDTO

Les DTO doivent contenir des données correctement typées.

Exemple pour une réservation :

salleId : int

responsable : string

email : string

motif : string

dateDebut : DateTimeImmutable

dateFin : DateTimeImmutable

Le tableau `$_POST` ne doit pas être transmis directement aux services.

Questions

1. Quelle différence existe entre DTO et modèle Eloquent ?
2. Pourquoi le DTO ne doit-il pas appeler `save()` ?
3. À quel moment transforme-t-on les chaînes en dates ?
4. Le DTO doit-il contenir la règle de chevauchement ?

Versionnement

Branche : feature/06-dto

Tag : v0.6.0

Étape 7 — Créer l'accès aux données

Travail demandé

Définissez des contrats pour les opérations nécessaires :

- SalleRepositoryInterface
- ReservationRepositoryInterface

Opérations minimales :

- lister les salles
- retrouver une salle
- enregistrer une salle
- lister les réservations
- retrouver une réservation
- rechercher un conflit
- enregistrer une réservation
- annuler une réservation

Créez des implémentations basées sur Eloquent.

Contrainte importante

Les contrôleurs ne doivent jamais contenir :

- Salle::query()
- Reservation::where(...)
- \$model->save()

Ces opérations doivent être isolées derrière les composants d'accès aux données.

Questions

1. Eloquent constitue-t-il déjà un accès aux données ?
2. Pourquoi ajouter un Repository au-dessus d'Eloquent ?
3. Cette abstraction est-elle toujours nécessaire ?
4. Quel avantage apporte-t-elle ?

Versionnement

Branche : feature/07-repositories

Tag : v0.7.0

Étape 8 — Implémenter les règles métier

Travail demandé

Créez :

- CreerReservationService
- AnnulerReservationService
- SalleIndisponibleException
- ReservationIntrouvableException

Le service de création doit :

1. retrouver la salle ;
2. vérifier qu'elle est active ;
3. vérifier que le début précède la fin ;
4. vérifier que la durée ne dépasse pas quatre heures ;
5. vérifier que la date est future ;
6. rechercher les chevauchements ;
7. créer la réservation ;
8. l'enregistrer ;
9. retourner le résultat.

Le service ne doit connaître :

- ni `$_POST` ;
- ni FastRoute ;
- ni les vues ;
- ni le conteneur.

Questions

1. Pourquoi ces règles ne sont-elles pas dans le contrôleur ?
2. Pourquoi le service dépend-il d'une interface de Repository ?
3. Quelle exception doit être levée en cas de conflit ?
4. Comment tester le service sans MySQL ?

Versionnement

Branche : feature/08-services

Tag : v0.8.0

Étape 9 — Créer les contrôleurs et les vues

Contrôleurs attendus

- SalleController
- ReservationController

Actions minimales

- SalleController::index()
- SalleController::show()
- SalleController::create()
- SalleController::store()
- SalleController::edit()
- SalleController::update()
- ReservationController::index()
- ReservationController::show()
- ReservationController::create()
- ReservationController::store()
- ReservationController::cancel()

Responsabilités de **store()**

1. lire les données HTTP ;
2. appeler le validateur ;
3. réafficher le formulaire en cas d'erreur ;
4. construire le DTO ;
5. appeler le service ;
6. rediriger après succès.

Vues attendues

- templates/layout/base.php
- templates/salle/index.php
- templates/salle/show.php
- templates/salle/form.php
- templates/reservation/index.php
- templates/reservation/show.php
- templates/reservation/form.php
- templates/error/404.php
- templates/error/405.php

Contraintes

- toutes les sorties dynamiques doivent être échappées ;

- les vues ne doivent appeler ni Eloquent ni le conteneur ;
- après un **POST** réussi, une redirection doit être effectuée ;
- les erreurs doivent apparaître près des champs concernés.

Versionnement

Branche : feature/09-interface-web

Tag : v0.9.0

Étape 10 — Configurer FastRoute

Routes attendues

Méthode	Chemin	Action
GET	/	Accueil
GET	/salles	Liste des salles
GET	/salles/create	Formulaire d'ajout
POST	/salles	Enregistrer une salle
GET	/salles/{id:\d+}	Détail
GET	/salles/{id:\d+}/edit	Modification
POST	/salles/{id:\d+}/edit	Enregistrer la modification
GET	/reservations	Liste

GET	<code>/reservations/create</code>	Formulaire
POST	<code>/reservations</code>	Créer
GET	<code>/reservations/{id:\d+}</code>	Détail
POST	<code>/reservations/{id:\d+}</code> <code>/cancel</code>	Annuler

Travail demandé

1. Déclarez les routes dans `routes/web.php`.
2. Créez le dispatcher FastRoute.
3. Supprimez la query string avant le dispatch.
4. Gérez les résultats :
 - `NOT_FOUND` ;
 - `METHOD_NOT_ALLOWED` ;
 - `FOUND`.
5. Pour une réponse `405`, ajoutez l'en-tête `Allow`.
6. Transmettez les paramètres dynamiques à l'action.
7. Ne construisez pas les contrôleurs dans le fichier des routes.

Format d'un handler

[

`ReservationController::class,`

`'show'`

]

Le routeur devra utiliser le conteneur pour obtenir le contrôleur correspondant.

Questions

1. Pourquoi FastRoute ne construit-il pas lui-même le contrôleur ?
2. Quelle différence existe entre `404` et `405` ?
3. Pourquoi contraindre `{id}` avec `\d+` ?
4. Quel composant doit interpréter le handler retourné ?

Versionnement

Branche : feature/10-router

Tag : v0.10.0

Étape 11 — Configurer PHP-DI

Travail demandé

Créez `config/container.php`.

Le conteneur doit connaître au minimum :

- Capsule\Manager
- SalleRepositoryInterface
- ReservationRepositoryInterface
- SalleValidator
- ReservationValidator
- CreerReservationService
- AnnulerReservationService
- SalleController
- ReservationController
- FastRoute\Dispatcher
- Application

Utilisez :

- l'autowiring pour les classes concrètes simples ;
- des définitions explicites pour les interfaces ;
- des factories pour les objets nécessitant une configuration.

Exemple attendu :

```
use function DI\autowire;
```

```
use function DI\factory;
```

```
use function DI\get;
```

```
return [
```

```
    SalleRepositoryInterface::class =>
```

```
        autowire(EloquentSalleRepository::class),
```

```

ReservationRepositoryInterface::class =>

autowire(EloquentReservationRepository::class),

Capsule\Manager::class =>

    factory(function (): Capsule\Manager {

        // Configuration de la connexion

    }),

];

```

Le fichier `public/index.php` doit devenir proche de :

```

<?php

declare(strict_types=1);

use App\Application;

use DI\ContainerBuilder;

require dirname(__DIR__) . '/vendor/autoload.php';

$builder = new ContainerBuilder();

$builder->addDefinitions(

    dirname(__DIR__) . '/config/container.php'

);

$container = $builder->build();

$application = $container->get(Application::class);

$application->run();

```

Interdiction

Les classes ne doivent pas recevoir `ContainerInterface` simplement pour rechercher leurs propres dépendances.

Code à éviter :

```
final class CreerReservationService  
  
{  
  
    public function __construct(  
  
        private ContainerInterface $container  
  
    ){  
  
    }  
  
}
```

Code attendu :

```
final class CreerReservationService{  
  
    public function __construct(  
  
        private SalleRepositoryInterface $salles,  
  
        private ReservationRepositoryInterface $reservations  
  
    ){  
  
    }  
  
}
```

Questions

1. Quelle différence existe entre injection et conteneur ?
2. Qu'est-ce que l'autowiring ?
3. Pourquoi les interfaces nécessitent-elles une définition ?
4. Pourquoi limiter `$container->get()` au point d'entrée ?
5. Quel anti-pattern apparaît si toutes les classes interrogent le conteneur ?

Versionnement

Branche : feature/11-container

Tag : v0.11.0

Étape 12 — Ajouter les tests

Tests unitaires obligatoires

Testez le service de création dans les situations suivantes :

1. réservation valide ;
2. salle inexistante ;
3. salle inactive ;
4. date de fin antérieure au début ;
5. durée supérieure à quatre heures ;
6. date passée ;
7. conflit avec une réservation ;
8. réservation voisine sans chevauchement.

Tests de validation obligatoires

Testez :

- une adresse électronique invalide ;
- un responsable vide ;
- une capacité négative ;
- un type de salle inconnu ;
- une date incorrecte.

Tests d'intégration minimaux

Testez :

- la création d'une salle avec Eloquent ;
- la relation salle/réervations ;
- la recherche de chevauchement ;
- l'annulation d'une réservation.

Contrainte

Les tests unitaires des services ne doivent pas nécessiter MySQL. Utilisez des doublures ou des implémentations en mémoire des repositories.

Versionnement

Branche : feature/12-tests

Tag : v0.12.0

Étape 13 — Finaliser l'application

Travail demandé

1. Ajoutez une mise en forme CSS simple.
2. Affichez les messages de succès et d'erreur.
3. Gérez proprement les exceptions.
4. Complétez le [README.md](#).
5. Complétez le [CHANGELOG.md](#).
6. Produisez le diagramme de classes.
7. Vérifiez l'installation depuis un dépôt fraîchement cloné.
8. Corrigez les anomalies.

Version finale

Branche : release/1.0.0

Tag : v1.0.0

10. Planning conseillé

Samedi matin — 4 heures

Objectif : obtenir une application initialisée et connectée à la base.

- initialisation Git ;
- création du projet Composer ;
- installation des dépendances ;
- configuration Eloquent ;
- création des tables ;
- création des modèles ;
- ajout des données initiales.

Version visée :

v0.4.0

Objectif : sécuriser les données entrantes.

- validation ;
- résultat de validation ;
- DTO ;
- tests des validateurs ;
- formulaires HTML simples.

Version visée :

v0.6.0

Samedi après-midi — 4 heures

Objectif : rendre les réservations fonctionnelles.

- repositories ;
- recherche des conflits ;
- services métier ;
- exceptions ;
- contrôleurs ;
- premiers tests unitaires.

Version visée :

v0.9.0

Dimanche matin — 3 heures

Objectif : brancher les composants techniques.

- configuration FastRoute ;
- gestion des réponses 404 et 405 ;
- configuration PHP-DI ;
- résolution des contrôleurs ;
- simplification de `index.php`.

Version visée :

v0.11.0

Dimanche après-midi — 3 heures

Objectif : fiabiliser et livrer.

- tests complémentaires ;
- correction des anomalies ;
- amélioration des vues ;
- documentation ;

- diagramme ;
 - historique Git ;
 - création de `v1.0.0`.
-

11. Scénarios de recette

Scénario 1 — Réservation valide

Salle : Salle B12

Début : demain à 10h00

Fin : demain à 12h00

Responsable : Awa Ndiaye

Email : awa.ndiaye@universite.sn

Motif : Cours d'architecture logicielle

Résultat attendu :

Réservation confirmée

Scénario 2 — Chevauchement

Réservation existante :

10h00 → 12h00

Nouvelle demande :

11h30 → 13h00

Résultat attendu :

La salle est indisponible pendant cette période.

Scénario 3 — Réservations voisines

Réservation existante :

10h00 → 12h00

Nouvelle demande :

12h00 → 14h00

Résultat attendu :

Réservation confirmée

Scénario 4 — Salle inactive

Résultat attendu :

Cette salle ne peut pas être réservée.

Scénario 5 — Durée excessive

Début : 08h00

Fin : 14h00

Résultat attendu :

Une réservation ne peut pas dépasser quatre heures.

Scénario 6 — Formulaire invalide

Responsable : vide

Email : adresse-invalid

Motif : TP

Résultat attendu :

- aucune insertion ;
 - affichage de toutes les erreurs ;
 - conservation des valeurs valides dans le formulaire.
-

Scénario 7 — URL inconnue

GET /inconnue

Résultat attendu :

404 — Page introuvable

Scénario 8 — Méthode non autorisée

DELETE /salles

Résultat attendu :

405 — Méthode non autorisée

La réponse doit contenir un en-tête `Allow`.

12. Livrables

Le dépôt doit contenir :

1. le code source complet ;
2. `composer.json` et `composer.lock` ;
3. `.env.example` sans secret ;
4. les migrations ou le schéma SQL ;
5. le script de données initiales ;
6. les tests ;
7. `README.md` ;
8. `CHANGELOG.md` ;
9. un diagramme de classes ;
10. une courte analyse des choix architecturaux.

Le `README.md` doit expliquer :

- les prérequis ;
- l'installation des dépendances ;
- la configuration de la base ;
- la création des tables ;
- l'ajout des données initiales ;
- le lancement du serveur ;
- l'exécution des tests.

13. Travail d'analyse

Dans un document **ARCHITECTURE.md**, les étudiants devront identifier et expliquer :

- MVC ;
- Front Controller ;
- Router ;
- Validator ;
- DTO ;
- ORM ;
- Active Record ;
- Repository ;
- Service ;
- injection par constructeur ;
- conteneur d'injection ;
- autowiring ;
- inversion de contrôle ;
- les cinq principes SOLID.

Pour chaque notion :

1. indiquez les classes concernées ;
2. expliquez son rôle ;
3. donnez un avantage ;
4. donnez une limite ou un risque ;
5. montrez un extrait représentatif du projet.

14. Bonus

Les étudiants peuvent ajouter :

- authentification des responsables ;
- rôle administrateur ;
- pagination ;
- recherche multicritère ;
- tableau de bord des salles les plus utilisées ;
- prévention CSRF ;
- middleware ;
- journalisation ;
- API JSON ;
- tests HTTP ;
- transactions lors des réservations ;
- protection contre deux réservations simultanées ;
- Docker ;

- intégration continue.

Le cœur obligatoire reste volontairement limité à deux ressources, [Salle](#) et [Reservation](#), afin que le projet soit réalisable sur un week-end tout en obligeant les étudiants à intégrer le routeur, la validation, Eloquent et le conteneur d'injection de dépendances.